

Análisis personal – Código de hacienda Isabella Gutiérrez Montoya

1. El sistema administra ciertos aspectos de la hacienda, principalmente aquellos relacionados con el mantenimiento y venta del ganado, esto lo logra llevando una trazabilidad de ciertos procesos como la venta de animales, su estado y el espacio físico donde se encuentran. Para esto le hace seguimiento al tipo de animal, su edad, su registro de vacunas, el peso, las ventas realizadas, la capacidad máxima y actual del potrero, entre otros.

Aunque existe un alto acoplamiento entre clases que hace que sean de alto riesgo o “importantes”, sus entidades principales son:

Res: Contiene los atributos de cada animal, como lo son su nombre, peso, edad y vacunas aplicadas, de esta clase posteriormente van a heredar, novillo, ternero y cebón.

Potrero: Contiene el método de añadir reces, crucial para el funcionamiento del negocio y define las condiciones de capacidad y almacenamiento de cada potrero, registrados con su identificación.

Vacuna: Lleva el registro de las vacunas que se le van a aplicar a los animales, fechas de vencimiento, entre otros

Usuario: Permite registrar e ingresar a los usuarios del sistema.

Venta: Coordina la venta de las reses

Lugares de alto costo para cambios:

1. Potrero. Aquí se realizan demasiadas tareas, por ende, cualquier modificación o extensión de la clase implicaría poner en riesgo muchas responsabilidades, hay demasiadas preocupaciones a la hora de validar la capacidad del potrero, la edad de las reses, su búsqueda entre otros y debido a su agrupación es difícil contemplarlas en su totalidad.

2. Zonas con if-else: en lugares como la validación de la res por debajo del peso mínimo, determinar el peso recomendado para la venta según el tipo de res y la verificación del esquema de vacunación, se perdió todo tipo de flexibilidad, no se puede hacer ningún cambio o modificación, violando el principio Open-Closed de SOLID, esto es sumamente riesgo por el crecimiento inminente del negocio.

3. Hay excepciones no implementadas y no contempladas, cualquier cambio en los hijos pueden quebrar el funcionamiento esperado inclumpliendo la sustitución LISKOV.

¿Cuál es la parte del código que en tu experiencia genera más fallos inesperados? ¿Cómo cuáles ?

Esto me permite priorizar fragmentos de código y me ayuda a contemplar más fallas que puedo estar omitiendo.